Introduction to Colab with Python (CS100)

CS 100, Intro to Programming (with Python), Catholic Polytechnic University, Summer 26

Table of Contents

1. Sources

- Introduction_to_Colab_with_Python_(CS100)_full_script.ipynb
- Starter code: is.gd/6nAJtZ

2. Objectives

By the end of this notebook, you should be able to:

- run a code cell in Colab
- edit code and run it again
- create a new code cell
- read simple Python code
- make small changes and observe what happens
- create a text cell with simple markdown

You'll also learn a few things about Python along the way.

This notebook is for practice, not for memorization.

Try things. Break things. Fix things.

3. What is Colab?

- Colab (full name: Google Colaboratory) is a so-called Jupyter notebook.
- "Jupyter" is a software developed for the languages Julia, Python, and R to enable "literate programming", i.e. writing code for humans rather than machines.

- Machines can totally live without text and explanations. They require only **syntactic** corrections (abide by the formal rules enforced by the parser), the rest is up to you. Example:

```
python3 -c "print('123' + 4)" 2>&1
```

```
Traceback (most recent call last):
  File "<string>", line 1, in <module>
TypeError: can only concatenate str (not "int") to str
```

- Besides syntactic errors there are also **semantic** (or "meaning") errors which gives the wrong results. Example:

```
a = 5
b = 10
print(a + b / 2)
print((a + b) / 2)
```

```
10.0
7.5
```

- The need for "literate programming" is also because coders don't like writing comments, so it is easy for code to be hard to read.
- In data science, notebooks are a way of exploring data via the formula

  ```
  data + stats + code -> story
  ```

  All four elements of this expression are contained in the notebook.
- Many programming books now come with Colab notebooks. Our textbook, also comes with such notebooks. Going through and running the code in the notebooks is a good way of reading it (writing the code yourself in a notebook is an even better way).

4. Why I don't use Colab

- Because I've got something better: Emacs + Org-mode
- Emacs was originally a text editor (published in 1985). It is extensible using Lisp. Because of this, it is totally customizable.
- Emacs users, over the past 40 years, have created a universe of packages for Emacs, and now it looks more like an operating system, and less like an editor.
- Org-mode is one of those packages, and allows smart notetaking and publishing. I develop all my content, code and text, in Emacs.
- Ask me for details if you're curious why and how you can try this.
- See my paper "Teaching data science with literate programming tools" (Digital, 2023).

5. The colab environment

Before we get on with Python and programming, a few words on the Colab environment.

Before you can edit the notebook that I gave you, you have to "**Save a copy in Drive**" (that is Google GDrive).

Each notebook has a title (top). You should change it now. CTRL+s will save your file. It will automatically be saved in a folder "**Colab Notebooks**". Check that now.

Before you see anything happening, you need to **Connect** to a "runtime instance", that is a CPU, which runs your code for you using Python (default), R (a statistical programming language), or Julia (a faster, newer language that sits between Python and R).

Once you connect you can see that you have **RAM** (Random Access Memory), and **Disk** space.

The left sidebar contains: A table of [notebook] contents (TOC), a link to the file list.

The TOC is driven by markdown commands like #, ##, etc. for a level 1,2 etc. headline.

When you open the link to the file list you can see that this is a full Linux operating system (a VM or Virtual Machine in fact).

Colab offers Google generative AI, Gemini: Click on the blue logo at the bottom of the screen to open a conversational pane with Gemini.

This is a most convenient way to quickly find out stuff. A traditional, and in many ways better (for learning) way is to use the language reference handbook or the `help()` function in the REPL.

Test it: Ask a simple question of the AI

```
How can I get help on a Python keyword?
```

Most likely, Gemini will immediately start messing with your notebook. Don't accept its suggestions but simply press **Cancel**.

Instead open the **Terminal** at the bottom. Through its shell program, this gives you access to the operating system, the software that controls all resources on this system. Now type at the prompt:

```
python3 --help
```

This gives you the options for the **python3** command.

When you enter only `python3` instead, you get to the REPL, the Read-Eval-Print-Loop of Python, where you can run any command, including calling the `help()` function. It opens a sub-system that contains the built-in help.

The last way to get help is to go to `python.org/doc/` - many different sources of information are listed here. Unlike with AI, these sources for help will not just make stuff up on the fly.

Before we go on: close the terminal, the AI panel and the side panel, and check out the options at the top to see what they mean.

Go to `Tools` > `Settings`, click on `Editor` and **uncheck** all sources.

For a more systematic exploration of Colab as a notebook, see this article: "Analying a bank failure with Colab" (May 8, 2026).

6. Demo overview

Open the sidebar again and look at the TOC.

The demo includes some Python basics, graphics, loops, random numbers, and types of errors known to Python.

7. Your first text cell

You won't find this in the code yet. Before `1. Your first code cell`, find the `+Text` button and add a new text cell:

````
## First text cell

- This is a list

1. This is a numbered list

`print()` is a function.

### This is a level 3 headline

This is a code verbatim block:
```
  if (True):
     print("True")
```
````

Now you can edit existing or enter new text cells wherever you like.

8. Your first code cell

Your first "code cell" is a line of **source code**.

Source (= original, human-readable) code means that the machine can turn it into a program that will run right here. You won't see how Python does it. When you click on the play button, or press `CTRL + ENTER`, it will run.

```
print("Hello, Python!")
```

```
Hello, Python!
```

Can you explain this code?

> A function `print` takes a string, `"Hello, Python!"` as argument and displays the string on the screen (also called `stdout`).

You can change the source code to get a different result.

Click on the code cell and `Copy` it, then paste it with CTRL + v below the text and run it.

```
print("Hello, Marcus!")
```

```
Hello, Marcus!
```

9. Python can do arithmetic

Computers are fancy calculators (actually, that's still all they are, from your gaming console over your phone to the snazzy AI).

Run the cells and check the results:

```
print(2 + 3)
print(10 - 4)
print(6 * 7)
print(8 / 2)
```

```
5
6
42
4.0
```

Can you explain this code?

> This time the argument of `print` is not a string but an **expression**. An expression is something that the computer can evaluate using numbers. In this case, we have four arithmetic expressions (no variables, just whole numbers) consisting of so-called **binary** (i.e. they have two operands) operators (addition, subtraction, multiplication, division).

What do you notice?

> All expressions except the last result in **integers**, whole numbers, without fractional part. The last one (division) results in a so-called floating-point number - a number with a decimal point. Not a very interesting number in this case, but it has a very different representation from an integer in the computer.
>
> You may also notice that there is space around the operators, which is ignored (you can test that). This is called **whitespace**.

There are more common binary operators.

```
print(2 ** 5)   # exponentiation
print(17 % 3)   # remainder (modulo)
print(17 // 3)  # integer (floor) division
```

```
32
2
5
```

What does the code mean?

> Three more operators are shown - exponentiation or power, computing the remainder of a division or modulo, and floor or integer division.
>
> Floor division `a//b` returns `floor(a/b)`, which is truncation toward zero for positive numbers (cutting of the fractional part. For negative numbers, `floor` rounds toward negative infinity, not zero: -17 // 3 = -6 (not -5). This is not so in C or Java.
>
> Python here preserves an **invariant**: (a//b)*b+(a%b)==a. So `%` is defined with `//` (remainder has the same sign as the divisor, not the dividend)
>
> These operations look all pretty simple though the algorithms (instruction sets) used by the computer to compute them fast, and the things you can do with them are interesting. For example:
>
> - The Euclidean algorithm for the greatest common denominator (GCD) uses the modulo operator.
> - For large integers, floor division and modulo use Knuth's Algorithm D.
> - Python uses exponentiation by squaring, which is also the core of cryptography (modular exponentiation in the RSA algorithm).
> - At the hardware level, all three reduce to **bitwise** operations
>
> Lastly, `#` is the **comment** sign - everything after it is ignored.

Why do you keep asking "what the code means"? Isn't it obvious?

> No, it is not obvious even though it seems that way. When you type your source code into the computer, or when you run a code cell, many things are happening at many different levels and places in the computer - none of them natural or easy to explain or understand.
>
> In order to begin to understand, you need to learn to articulate (at least in your mind), with utmost clarity (that means: no ambiguity, no shrugging "I don't know" or "I'm not sure") two things:
>
> 1. What did you mean to write when you wrote the code?
> 2. What did the computer see when you wrote it?
>
>   With these two bits of information, you'll be able to solve problems of computing (not of mathematics, not of "critical" thinking or whatever) - problems that have to do with making the computer do your scientific work.

Before running the next cell, predict the result:

```
print(3 + 4 * 5) # prints 23 = 3 + 20
```

```
23
```

What happens if you use parentheses: `print((3+4)*5)`?

```
print((3 + 4) * 5) # prints 7 * 5 = 35
```

```
35
```

Explain this?

> Evaluating expressions with different operators follows rules of precedence - the well-known PEMDAS rule: Parentheses > Exponentiation > Multiplication > Division > Addition > Subtraction.

10. Words and numbers are different

Python distinguishes between text and numbers - at the user level.

At the machine level, only numbers are known: The letter `A` for example corresponds to the ASCII code `65`:

```
print(ord('A')) # converts character string to ASCII number
print(chr(65))  # converts ASCII number to character string
print(type(ord('A'))) # it's an int[eger]
print(type(chr(65)))  # it's a str[ing]
```

```
65
A
<class 'int'>
<class 'str'>
```

Compare the results from these cells:

```
print(33)
print("33")
```

```
33
33
```

No difference in the display, but as you will see, the `type` is different:

```
print(type(33))   # integer
print(type("33")) # string
```

```
<class 'int'>
<class 'str'>
```

Introducing operations changes the picture:

```
print(2 + 3)
print("2" + "3")
```

```
5
23
```

Why are these results different?

> `2 + 3` is an arithmetic operation, while `"2" + "3"` only uses the same symbol but the operator executes a **concatenation** of two strings.

11. Variables

A **variable** stores a **value** so that we can use it later.

```
name = "Adam"
print(name)
```

```
Adam
```

```
x = 11
y = 22
print( x + y)
```

```
33
```

Try this:

- Change the values of `name`, `x`, and `y`.
- Run the cells again.
- Create a new variable called `course`.
- Store the name of this course in it.

Solution:

```
name = "Eve"
print(name)
x = 2
y = 10
print(x + y)
```

```
course = "Introduction to programming"
print(course)
print("Course name: " + course)
```

```
Eve
12
Introduction to programming
Course name: Introduction to programming
```

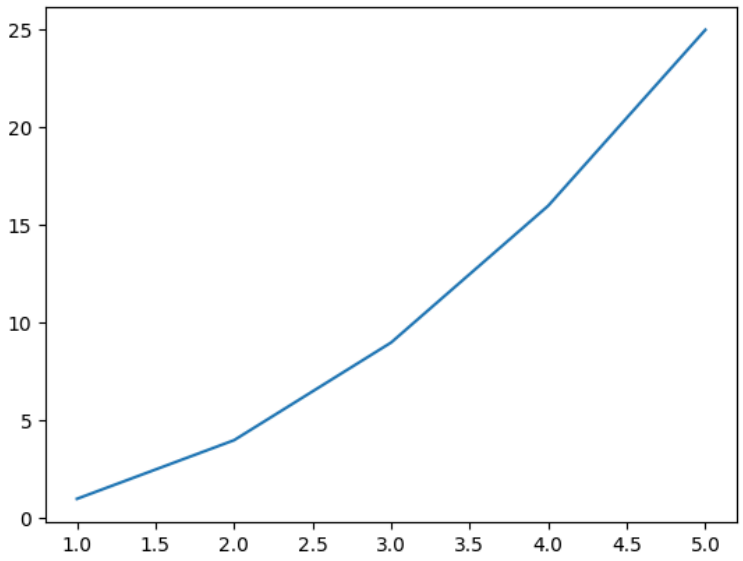
12. A small "fancy" example

Python can make simple graphics.

```
import matplotlib.pyplot as plt # import library for graphics

x = [1,2,3,4,5]    # list
y = [1,4,9,16,25]  # another list

plt.plot(x,y)  # plot y = f(x)
plt.savefig("../img/plt.png") # save the plot in a PNG file
```



What's going on here?

- The first line imports a **library** so that we can plot graphics.
- The variables `x` and `y` are not just numbers but lists.
- The values in the lists form `(x,y)` pairs: `(1,1)`, `(2,4)`, `(3,9)` etc.
- The `plot()` function draws a line between these pairs.
- The `savefig()` function saves the plot as a PNG file.
- In Colab, `show()` function displays the plot in the notebook.

Try this:

- Change the numbers in `y` and run the cell again: How would you describe the results?

```
y2 = [1,2,3,4,5]      # linear function y = x
y3 = [5,4,3,2,1]      # negative slope y = -x + 6
y4 = [1,8,27,64,125]  # cubic function y = x**3
```

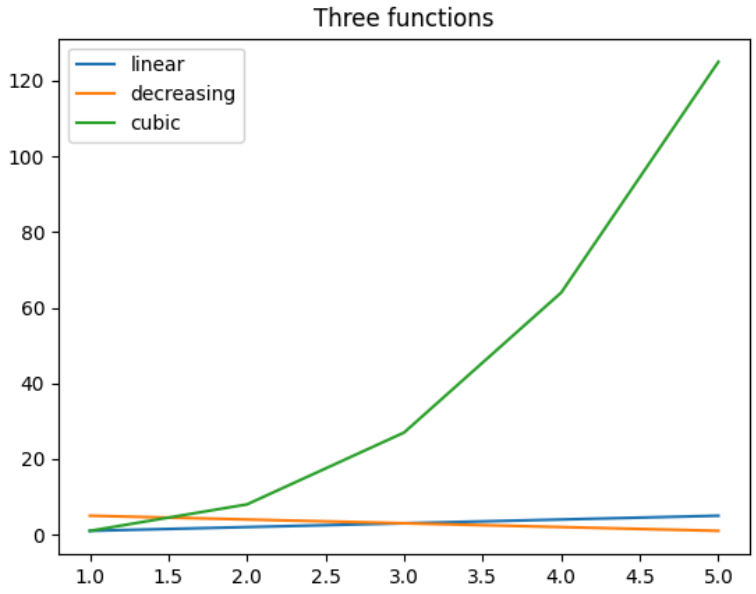
- Show all three functions in the same plot:

```
y2 = [1,2,3,4,5]
y3 = [5, 4, 3, 2, 1]
y4 = [1, 8, 27, 64, 125]

plt.plot(x,y2,label="linear")
```

```
plt.plot(x,y3,label="decreasing")
plt.plot(x,y4,label="cubic")

plt.title("Three functions")
plt.legend()
plt.savefig("../img/plt2.png")
```



This plot adds two more plot **customization** functions:

- `title()` adds a title to the plot.
- `legend()` adds a legend to the plot (using `label`).

13. A fancier plot with real data

Here we download real data from a public open data portal and plot them.

We download a public dataset and plot:

- Total New York City water consumption
- Per-capita water use

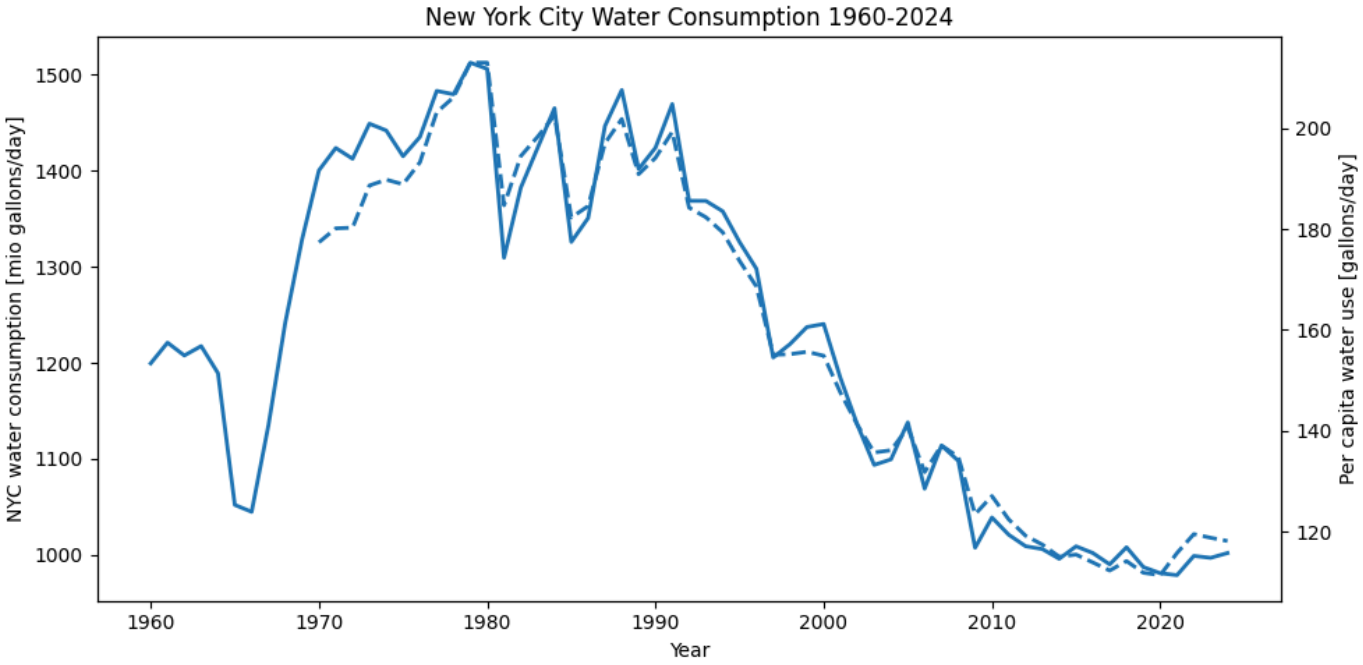
Run the cell and inspect the graph.

```
import pandas as pd
import matplotlib.pyplot as plt

url="https://data.cityofnewyork.us/api/views/ia2d-e54m/rows.csv?accessType=DOWNLOAD"
df = pd.read_csv(url)

fig, ax1 = plt.subplots(figsize=(10,5))
ax1.plot(df["Year"],
         df["NYC Consumption(Million gallons per day)"],
         linewidth=2)
ax1.set_xlabel("Year")
ax1.set_ylabel("NYC water consumption [mio gallons/day]")
ax1.set_title("New York City Water Consumption 1960-2024")

ax2 = ax1.twinx()
ax2.plot(df["Year"],
         df["Per Capita(Gallons per person per day)"],
         linestyle="--",
         linewidth=2)
ax2.set_ylabel("Per capita water use [gallons/day]")
fig.tight_layout()
plt.savefig("../img/NYCplot.png")
```



Alternative data source: https://is.gd/CcdfoQ

Let's unpack the code:

- `pandas` is another library, for manipulation of tabular data.
- The `url` stores an internet address.
- `df` stores the tabular data as a **data frame**.
- The data are formatted in CSV (comma-separated values).
- `subplots()` prepares a canvas and x/y axes.
- `plot()` selects two columns from the table.
- `twinx()` shares the x-axis values between both plots.
- `tight_layout()` tightens the figure which contains a lot of data.

What do you see in the plot? How would you interpret this plot?

Observe:

- Consumption rises from 1960 to about 1980 (1,500 mio gal/day)
- From 1980 sustained decline to 1,000 mio gal/day
- NYC population grew from 7.78 mio (1960) to 8.48 mio (2024)
- Dip in 1965: drought (external shock to the system)
- Zigzags 1980-1990: drought restrictions and their relaxation.
- Both curves rise and fall together: population growth not driver.
- Post 2000 decline is structural: metering, appliance efficiency, and de-industrialization.

Try this:

- Plot only one of the two lines (comment one of the `plot` commands)
- Change the title from "Over Time" to "1960-2024"
- In a new code block, print `df.head()` to inspect the data
- Change `figsize=(10,5)` to another size - e.g. (20,5)

14. Alternative example: Abortion statistics (bonus assignment)

As a challenge, use the following data to plot the legal induced abortions in the US between 1973 and 2000 according to CDC data:

```
# Source: CDC Abortion Surveillance Reports (MMWR), 1973-2022
# Rate = abortions per 1,000 women aged 15-44
year         = [1973,1976,1979,1982,1985,1988,1991,1994,1997,2000,
                2003,2006,2009,2012,2015,2018,2019,2020,2021,2022]
rate         = [16.3,21.7,25.4,28.8,28.0,27.3,26.3,24.1,20.7,21.3,
                21.1,19.4,19.6,16.9,14.6,11.6,11.4,11.2,11.4,11.2]
total_1000s  = [616, 988,1251,1574,1589,1591,1557,1268,1186,857,
                848, 846, 789, 699, 638, 619, 625, 620, 625, 613]
```

Try this:

1. Create a `fig, ax1` pair of `figsize (10,5)` with `plt.subplots`
2. Plot the abortion `rate` as a function of the `year`
3. Set the `linewidth` to 2 and the `label` to `"rate"`
4. Set the x label of `ax1` to `"Year"`
5. Set the y label of `ax1` to `"Rate per 1,000 women aged 15-44"`
6. Set the title to `"Legal induced abortions, United States (CDC)"`
7. Share the x axis between `ax1` and `ax2` with `twinx()`
8. Plot the total number of abortions in 1000s as function of `year`
9. Use `linestyle` dashed `"--"`, `linewidth` 2, and set `label` to `"total"`
10. Set the y label of `ax2` to `"Total reported (thousands)"`
11. Tighten the layout with `fig.tight_layout()`
12. Save the plot to a file `"cdc.png"`

- This is a bonus assignment: you get extra points when you submit it.
- Bonus assignments can be done any time before the last class.
- Use the examples from the Colab and Python demo in class (see GitHub).
- Do not use AI for this exercise: challenge yourself to type the code in yourself.
- Submit the URL to your Colab notebook solution in the textbox to the exercise

15. Repetition with a loop

Data usually come in bulk, as a result of measurements, and they are stored in collections or containers. To process many data points in sequence, programming languages "loop" over the data.

In the following loop, also called a `for` loop because it is triggered by that keyword, a variable `i` serves as loop variable. It ranges over five data points only. They are generated by the `range()` function.

Then, for every data point, or value of the range, a message is printed:

```
for i in range(5):
    print("Python is running", i)
```

```
Python is running 0
Python is running 1
Python is running 2
Python is running 3
Python is running 4
```

You notice that, like C, Python starts counting at 0 and not at 1, and that the `range()` argument (5) is an upper bound and not included.

Try this:

- Change `range(5)` to `range(3)`
- Change the variable inside `print(...)`

```
for i in range(3):
    print("Quadratic loop values:", i, "square is", i**2)
```

```
Quadratic loop values: 0 square is 0
Quadratic loop values: 1 square is 1
Quadratic loop values: 2 square is 4
```

16. Random numbers

Python cannot generate true random values - it's `random` module uses the Mersenne Twister, a pseudo-random number generator, which is deterministic given a seed. This means that its numbers can be computed using a function. There are Python libraries that harvest external sources getting closer to true random number generation.

What would a "true random number" be? It would not be predictable. But pseudo random numbers can be regenerated identically for a given seed.

This code uses the `for` loop to generate a `range` of 5 integer random numbers between 1 and 10.

```
import random

for i in range(5):
    print(random.randint(1,10))
```

```
3
7
6
5
1
```

Try this:

- Change the range of random numbers.
- Add `random.seed(325)` and see what happens.

```
import random
random.seed(325) # 325 AD Council of Nicea (resolved the Arian heresy)
for i in range(5):
    print(random.randint(1,10))
```

```
9
3
10
2
2
```

17. Errors are normal

This cell (run on the shell) contains a so-called `SyntaxError`:

```
python3 -c 'print("Hell)' 2>&1 # SyntaxError
```

```
  File "<string>", line 1
    print("Hell)
          ^
SyntaxError: unterminated string literal (detected at line 1)
```

There are other errors: E.g. `TypeError`, `NameError`, `ValueError`:

```
python3 -c 'int("A")' 2>&1 # ValueError
```

```
Traceback (most recent call last):
  File "<string>", line 1, in <module>
ValueError: invalid literal for int() with base 10: 'A'
```

```
python3 -c 'print(x)' 2>&1 # NameError
```

```
Traceback (most recent call last):
  File "<string>", line 1, in <module>
NameError: name 'x' is not defined
```

```
python3 -c 'print("age:" + 40)' 2>&1 # TypeError
```

```
Traceback (most recent call last):
  File "<string>", line 1, in <module>
TypeError: can only concatenate str (not "int") to str
```

18. Create your own code cell

Add a new code cell and do all of this:

- Print your name
- Print the result of `7 * 8`
- Assign the result of `7 * 8` to a variable `x` and print it

```
print("Marcus Birkenkrahe")
print(7 * 8)
x = 7 * 8
print(x)
```

```
Marcus Birkenkrahe
56
56
```

19. Challenges (home assignment)

1. Print your favorite number:

```
print(1/137) # the fine-structure constant (Sommerfeld's constant)
```

```
0.0072992700729927005
```

2. Create two variables with numbers and print their sum:

```
x = 33
y = 12
print(x + y)
```

```
45
```

3. Print the numbers 1 through 5 using a loop

```
for i in range(5):
    print(i + 1)
```

```
1
2
3
4
5
```

4. Generate three random numbers between 1 and 20 using a loop:

```
import random

for i in range(3):
    print(random.randint(1,20))
```

```
6
12
8
```

5. Make a graph with your own x and x values:

```
import matplotlib.pyplot as plt

x = ["Pi", "Euler", "Sommerfeld"]
y = [3.14, 2.71, 1/137]

plt.plot(x, y, marker="o", linestyle="--")
plt.title("Pi, Euler number, and Sommerfeld's constant")
plt.savefig("../img/sommerfeldt.png")
```

These constants are also built-in:

```
import math
from scipy.constants import fine_structure

print(math.pi) # Pi
print(math.e)  # exp(1) = e
print(fine_structure)
```

```
3.141592653589793
2.718281828459045
0.0072973525643
```

Author: Marcus Birkenkrahe with ChatGPT
Created: 2026-05-22 Fri 21:50